

Lab 3 - Forward Kinematics/Coordinate Transformations*

EECS/ME/BIOE C106A/206A Fall 2026

Goals

By the end of this lab you should be able to:

- Identify the UR7e's joints and interpret its published joint states
- Visualize the UR7e and its coordinate frames using RViz
- Compute the forward kinematics of the UR7e from its joint configuration
- Use `tf2` to query coordinate transformations in a ROS2 node
- Compare direct joint-space control with motion planning using MoveIt

Contents

0	Initial Setup	2
1	Introduction to the UR7e	5
1.1	Connecting to the Robot	5
1.2	Moving the UR7e with the Keyboard	5
1.3	Observing the Joint State	6
1.4	Freedrive Mode	6
1.5	RViz	6
2	Forward Kinematics	6
2.1	Robot Descriptions and URDF	7
2.2	Using Your Homework 2 Code	7
2.3	Getting the Current Joint Angles	8
2.4	Forward Kinematics Node	8
3	TF and Coordinate Frames	9
3.1	Writing a TF Listener	10
3.2	Comparing TF and Forward Kinematics	10
4	Controlling the UR7e	11
4.1	Choosing Target Configurations	11
4.2	Direct Joint Control	11
4.3	Motion Planning with MoveIt	11
4.4	Conclusion	13
5	Before you Leave	13

*Redeveloped by Aarsh Shroff, Reid Dye, Fall 2026. Modified by: Ishan Dogra and Daniel Municio, Fall 2025; Mingyang Wang, Fall 2023; Ravi Pandya, Nandita Iyer, Phillip Wu, and Valmik Prabhu, Fall 2018; David Fridovich-Keil and Laura Hallock, Fall 2017; Dexter Scobee and Oladapo Afolabi, Fall 2016; Victor Shia and Jaime Fisac, Fall 2015. Developed by Aaron Bestick, Austin Buchan, Fall 2014.

0 Initial Setup

Before beginning this lab, we need to standardize everyone's development environment. **This setup is required. Please follow every step carefully before continuing.**

Lab Computer and Partner Setup

Beginning with this lab, you may no longer use VM/RDP on your personal computer. All remaining labs must be completed using the lab computers.

You should also choose a partner that you will work with for the remainder of the course. Pick one partner's account to use on the lab computer, log into that account, and complete the following setup together.

Also, if you haven't pushed your current lab 2 code yet, please do so before continuing with these instructions.

Create a Fresh ROS2 Distrobox

Open a terminal on the lab computer. Check which Distroboxes currently exist:

```
distrobox list
```

Remove each existing Distrobox using:

```
distrobox rm -f <container_name>
```

Then create the full ROS2 container using:

```
inst-containers-setup  
~ee106a/create-ros2-container
```

Enter the new container with:

```
distrobox enter ros2
```

Set up SSH

Configure your Git identity:

```
git config --global user.name "Your Name"  
git config --global user.email "your_email@example.com"
```

Next, generate an SSH key pair:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

When prompted for a location to save the key, press **Enter** to use the default location. Just press **Enter** twice more when prompted to enter a passphrase. Then start the SSH agent and add your newly created key:

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

Now print your **public** SSH key:

```
cat ~/.ssh/id_ed25519.pub
```

Copy the **entire** output. Then, open your browser and go to:

GitHub → Settings → SSH and GPG keys → New SSH key

Name the key **EECS 106A Lab Machine**, set it as an **Authentication Key**, paste your public key into the key field, and save it.

Finally, verify that SSH authentication is working:

```
ssh -T git@github.com
```

The first time you connect, you may be asked whether you trust GitHub's host key. Type **yes**. If everything is configured correctly, you should see a message similar to:

```
Hi USERNAME! You've successfully authenticated,  
but GitHub does not provide shell access.
```

Clone Your Personal Repository

Inside the **ros2** Distrobox, clone the private GitHub repository you created during the previous labs:

```
cd ~  
git clone git@github.com:GITHUB_USERNAME/YOUR_REPO.git ros_workspaces  
cd ~/ros_workspaces
```

Replace **GITHUB_USERNAME** with your GitHub username and **YOUR_REPO** with the name of your personal GitHub repository. **Next, go on GitHub and share your personal repository with your partner.**

Add the Fall 2026 Starter Repository

Next, add the Fall 2026 course repository as a second remote:

```
git remote add starter https://github.com/ucb-ee106/106a-student-labs-starter.git  
git fetch starter
```

Your repository now has two remotes:

- **origin**: your personal GitHub repository
- **starter**: the official course starter repository

Throughout the rest of the course, you will use **starter** to retrieve the newest version of each lab while continuing to push your completed work to **origin**.

Clean Up Your Repository

Due to mistakes on our end, your personal repository accidentally contains outdated versions of the labs. Before continuing, remove those directories from your repository:

```
git rm -rf --ignore-unmatch -- lab3 lab3_ur7e lab4 lab5 lab6 lab7 lab8  
git restore --source=starter/main --staged --worktree -- lab3 lab4  
git commit -m "Update Labs 3 and 4 for Fall 2026"  
git push origin main
```

Set Up an IDE

For the remainder of the course, please use **Visual Studio Code** (or another IDE) for development rather than editing directly with **vim**, **nano**, or **EMACS**. Open your lab workspace in VS Code and install the following extensions:

- Python — Microsoft
- C/C++ Extension Pack — Microsoft
- CMake Tools — Microsoft
- XML — Red Hat
- YAML — Red Hat

Along with that, as you work, use **Ctrl+Shift+T** to open new terminal tabs instead of creating many separate terminal windows. You may also use the integrated terminal in VS Code.

Checkpoint 0

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. Before proceeding with the lab, you must show a TA that:

- You set up your Distrobox with the **ros_workspaces** folder with both the origin and starter remotes, and SSH setup
 - VS Code is set up for development
 - You have completed the **Robot Usage Quiz** on Gradescope
-

1 Introduction to the UR7e

In this lab, we will work with the **UR7e**, a robotic manipulator manufactured by **Universal Robots**. The UR7e is a **6-degree-of-freedom (6-DOF) collaborative robot arm** with six independently actuated revolute joints. The six joints of the UR7e are:

1. shoulder_pan_joint
2. shoulder_lift_joint
3. elbow_joint
4. wrist_1_joint
5. wrist_2_joint
6. wrist_3_joint

Together, these joints determine the position and orientation of the robot's end effector. Throughout this lab, we will explore how ROS represents the robot's joint configuration and how these joint angles determine the pose of the robot.

1.1 Connecting to the Robot

Enter the `lab3` workspace and build and source the workspace as usual. To connect ROS2 to the UR7e, run:

```
ros2 run ur7e_utils enable_comms
```

Leave this node running for as long as you are using the robot.

1.2 Moving the UR7e with the Keyboard

Let's first get familiar with how the robot's joints move. In a new terminal, run:

```
ros2 run joint_control validate_trajectory
```

Keep this running throughout the lab. It checks to make sure you keep the joints in safe positions! Next, reset the robot position with:

```
ros2 run ur7e_utils tuck
```

Next, run:

```
ros2 run ur7e_utils keyboard_controller
```

Follow the instructions printed in the terminal to move the individual joints of the UR7e. As you control the robot, make sure one partner keeps their hand on the E-stop button.

Move each joint a small amount in both directions and observe the physical robot. As you do this, pay attention to:

- Which part of the robot each joint rotates
- The axis about which each joint rotates
- How moving joints closer to the base affects the rest of the arm
- How the wrist joints affect the orientation of the end effector

You should be able to identify all six joints on the physical robot.

1.3 Observing the Joint State

While you move the robot, ROS continuously publishes information about its current joint configuration on the topic:

```
/joint_states
```

View the messages being published on this topic (remember the command from Lab 1!!). You should see messages containing fields such as:

```
name:  
position:  
velocity:  
effort:
```

For now, focus on **name** and **position**. The **name** array identifies each joint, while the corresponding entry in **position** gives that joint's current angle in **radians**.

Keep `/joint_states` visible while using the keyboard controller. Move one joint at a time and observe which joint angle changes.

The important idea is that, at any instant, the configuration of the UR7e can be represented by six joint angles:

$$\theta = [\theta_1 \quad \theta_2 \quad \theta_3 \quad \theta_4 \quad \theta_5 \quad \theta_6]^T$$

1.4 Freedrive Mode

The UR7e also supports **freedrive**, a mode in which the robot's joints can be moved manually by applying force to the arm. **Ctrl+C** the `keyboard_controller` node and run:

```
ros2 run ur7e_utils freedrive
```

Please make sure to move the robot slowly and gently! If you move it too fast, the robot will trigger its safety lock and we will need to reset its state. Watch the `/joint_states` topic to see how they change.

1.5 RViz

ROS also provides a visualization tool called **RViz2** that allows us to view the robot's model and state in 3D. To launch RViz with the UR7e, run:

```
rviz2 -d /opt/ros/humble/share/ur_description/rviz/view_robot.rviz
```

You should see a 3D model of the UR7e appear in RViz. While RViz is open, use the keyboard controller or freedrive to move the physical robot. Observe how the robot model in RViz updates to match the real robot. Take a few minutes to get familiar with the RViz interface.

2 Forward Kinematics

Now that you have seen how the UR7e's joints move and how its state can be visualized in RViz, we will construct our own **forward kinematics** implementation.

Recall from lecture and Homework 2 that the forward kinematics of a robotic manipulator can be written using the product of exponentials:

$$g_{st}(\theta) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} \dots e^{\hat{\xi}_6 \theta_6} g_{st}(0).$$

There are three main pieces of information in this expression:

- θ_i : the current angle of joint i
- ξ_i : the twist associated with joint i
- $g_{st}(0)$: the transformation from the base frame to the final frame when all joint angles are zero

A `forward_kinematics` package has been provided. In this section, you will complete the provided starter code in this package to build your forward kinematics node. **Please read through the entire section before coding.**

2.1 Robot Descriptions and URDF

ROS represents the structure of a robot using a **URDF (Unified Robot Description Format)**. A URDF describes a robot in terms of its links, joints, and the geometric relationships between them. For this lab, we have provided a simplified version of the URDF for the UR7e at:

```
urdf/ur7e_flattened.urdf
```

The joint entries in the file look like:

```
<joint name="shoulder_pan_joint" type="revolute">
  <parent link="base_link"/>
  <child link="shoulder_link"/>
  <origin xyz="0.0 0.0 0.1625"/>
  <axis xyz="0.0 0.0 1.0"/>
</joint>
```

The **parent** and **child** fields describe which links are connected by the joint. The **origin** field gives a point q_i on the joint's axis of rotation, expressed in the **base_link** frame. The **axis** field gives the joint's axis of rotation ω_i , also expressed in the **base_link** frame.

Take a minute to look through the URDF file and identify the six revolute joints of the robot.

For a revolute joint, recall that

$$v_i = -\omega_i \times q_i$$

and therefore the corresponding twist is

$$\xi_i = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix}.$$

The provided file also contains the robot's zero-configuration transformation, $g_{st}(0)$. This transformation describes the pose of **wrist_3_link** relative to **base_link** when all six joint angles are zero.

It is represented using a rotation matrix R and translation vector p :

$$g_{st}(0) = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}.$$

Together, the six joint twists and $g_{st}(0)$ provide the fixed kinematic information needed to evaluate the product-of-exponentials forward kinematics map.

2.2 Using Your Homework 2 Code

Copy your completed `kin_func_skeleton.py` from Homework 2 into the starter `kin_func_skeleton.py` file.

In particular, you will reuse your implementation of

```
forward_kinematics(xi, theta)
```

which computes

$$e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} \dots e^{\hat{\xi}_6 \theta_6}.$$

Recall that this function computes only the product of exponentials. To obtain the complete forward kinematics map, you must also multiply by $g_{st}(0)$:

$$g_{st}(\theta) = FK(\xi, \theta) g_{st}(0).$$

where $FK(\xi, \theta)$ is the value returned by your `forward_kinematics(xi, theta)` function from Homework 2.

2.3 Getting the Current Joint Angles

The twists and $g_{st}(0)$ describe the fixed geometry of the robot. The joint angles θ_i , however, change whenever the robot moves.

As you saw earlier, the UR7e continuously publishes its current joint states on

```
/joint_states
```

Your forward kinematics node will subscribe to this topic and extract the six joint angles from each `JointState` message.

Remember that the ordering of the joints in `/joint_states` may not necessarily match the ordering used by your forward kinematics code. Use the joint names in the message to ensure that each angle is matched with the correct twist.

2.4 Forward Kinematics Node

Starter code has been provided in:

```
forward_kinematics_node.py
```

The starter code loads `urdf/ur7e_flattened.urdf` and extracts the values q_i , ω_i , R , and p described above. It also creates a subscriber to the `/joint_states` topic.

Your job is to use this information to construct the UR7e's forward kinematics map and evaluate it using the robot's current joint angles.

Task 1: Complete `forward_kinematics_node.py` so that it computes the current transformation from `base_link` to `wrist_3_link`.

First, complete `load_kinematics_description()`. Using the q_i and ω_i values extracted from the robot description:

1. Compute the twist corresponding to each of the six joints:

$$\xi_i = \begin{bmatrix} -\omega_i \times q_i \\ \omega_i \end{bmatrix}.$$

2. Combine the six twists into a 6×6 NumPy array, with one twist in each **column**. This is the format expected by your Homework 2 `forward_kinematics` function.
3. Construct the zero-configuration transformation using the provided rotation matrix R and translation vector p :

$$g_{st}(0) = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}.$$

Hint: NumPy's `np.cross` and `np.concatenate` functions may be useful when constructing each twist. Once you have all six twists, consider how `np.column_stack` can be used to arrange them in the expected format.

Next, complete `joint_state_callback()`. Whenever a new `JointState` message is received:

1. Extract the current angles of the six UR7e joints.
2. Arrange the angles in the same order as the twists in `JOINT_NAMES`.
3. Use your Homework 2 `forward_kinematics` function together with the twists and $g_{st}(0)$ you constructed above to compute the full forward kinematics map:

$$g_{st}(\theta) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} \dots e^{\hat{\xi}_6 \theta_6} g_{st}(0).$$

4. Print the resulting 4×4 homogeneous transformation matrix.

Hint: Do not assume that the order of the joints in a `JointState` message matches the order in `JOINT_NAMES`. The `name` and `position` arrays correspond element-by-element. One useful approach is to first associate each joint name with its corresponding position, then retrieve the positions in the order specified by `JOINT_NAMES`.

When your node is complete, run it while moving the UR7e using the keyboard controller or freedrive. Observe how the resulting transformation changes as the robot moves.

Your final output represents the pose of `wrist_3_link` relative to `base_link` as computed by your own forward kinematics implementation.

Checkpoint 1

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Explain how the URDF, `/joint_states` topic, and your Homework 2 forward kinematics code work together in your complete FK node (**Make sure you understand the FK node thoroughly!**)
 - Explain what the resulting $g_{st}(\theta)$ transformation represents
-

3 TF and Coordinate Frames

So far, RViz has shown us the geometry of the UR7e as it moves. We can also use RViz to visualize the **coordinate frames** attached to each link of the robot.

ROS manages these coordinate-frame relationships using **TF**. TF maintains a tree of coordinate frames and the transformations between them. For the UR7e, this includes frames attached to the base, shoulder, wrist, and other links of the robot.

With RViz open, click **Add** in the Displays panel and add a TF display. You should now see coordinate axes appear throughout the robot. Each set of axes represents a coordinate frame associated with one of the robot's links.

Expand

```
TF -> Frames
```

in the Displays panel. You can enable or disable individual frames to make the visualization easier to interpret. In particular, locate the frames

```
base_link
wrist_3_link
```

and observe how the pose of `wrist_3_link` changes as you move the robot.

The transformation from `base_link` to `wrist_3_link` describes the pose of the robot's wrist relative to its base. This is the same transformation that you computed using forward kinematics in the previous section.

TF can also be queried directly from the command line. Run:

```
ros2 run tf2_ros tf2_echo base_link wrist_3_link
```

This command continuously prints the transformation from `base_link` to `wrist_3_link`, including both its translation and orientation.

3.1 Writing a TF Listener

The `tf2_echo` command uses ROS's `tf2` library to query transformations between coordinate frames. We will now reproduce this functionality in our own ROS2 node.

A `tf2_ros.Buffer` stores the transforms currently known by TF:

```
tf_buffer = tf2_ros.Buffer()
```

A `tf2_ros.TransformListener` listens for TF updates and stores them in the buffer:

```
tf_listener = tf2_ros.TransformListener(tf_buffer, self)
```

Once the buffer contains the requested frames, you can query the transform between them using:

```
tf_buffer.lookup_transform(  
    target_frame,  
    source_frame,  
    rclpy.time.Time()  
)
```

The returned value is a `geometry_msgs/TransformStamped` [geometry_msgs/TransformStamped](#) message containing the translation and rotation between the two frames.

Task 2: Complete the provided `tf_listener.py` node in the `tf_listener` package.

You will need to:

1. Create a `tf2_ros.Buffer` and `tf2_ros.TransformListener` in the node constructor.
2. Implement `print_transform()` to query and print the requested transform
 - Use the provided `quaternion_to_rotation_matrix()` helper to convert the returned quaternion into a rotation matrix before printing the result.
 - Put a `try-except` around the body of the function to catch TF errors.

Once your node is complete, run it to query the transformation from `base_link` to `wrist_3_link`:

```
ros2 run tf_listener tf_listener base_link wrist_3_link
```

3.2 Comparing TF and Forward Kinematics

You now have two independent ways of determining the pose of `wrist_3_link` relative to `base_link`:

1. Your forward kinematics implementation, which computes the transformation from the robot's joint angles.
2. TF, which provides the transformation maintained by ROS.

Run your forward kinematics node and TF listener at the same time.

The translation and rotation produced by your forward kinematics implementation should agree with the transformation reported by TF, up to small numerical differences.

Checkpoint 2

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Explain what TF represents and how transforms between coordinate frames are obtained
 - Walk through your TF listener code and explain how it queries the transform from `base_link` to `wrist_3_link`
 - Move the robot in multiple poses and demonstrate that your FK code and TF code produce the same output
-

4 Controlling the UR7e

So far, we have observed the state of the UR7e and computed its pose using forward kinematics and TF. We will now begin commanding the robot ourselves.

In this section, you will move the UR7e between the same two joint configurations using two different approaches:

1. Direct joint trajectory control
2. Motion planning with MoveIt

This will let us compare directly commanding the robot's joints with using a higher-level motion-planning framework.

4.1 Choosing Target Configurations

Before writing any control code, use the keyboard controller to place the UR7e into two safe configurations of your choice. We will refer to these as **Pose A** and **Pose B**.

For each pose, use the `/joint_states` topic to record the six joint angles. Keep track of both the joint names and their corresponding positions, since you will use these same two configurations throughout the rest of the lab.

4.2 Direct Joint Control

We will first move the UR7e by directly commanding a desired joint configuration.

ROS represents joint trajectories using the `trajectory_msgs/JointTrajectory` message type. A `JointTrajectory` specifies which joints are being controlled and contains one or more `JointTrajectoryPoint` messages describing desired joint positions over time.

Task 3: Complete the `publish_trajectory()` function in the `joint_controller.py` node in the `joint_control` package. Your function should:

- Create a `JointTrajectory` message with the joint names
- Add a `JointTrajectoryPoint` containing the target positions from `self.joint_angles`
- Set the velocity to an array of 6 zeros and `time_from_start` as 5
- Publish the completed trajectory to `/joint_trajectory_validated`

Once your controller is complete, use the joint angles you recorded earlier to move the robot from Pose A to Pose B. If nothing happens when you run your code, make sure `validate_trajectory.py` is running.

4.3 Motion Planning with MoveIt

Direct joint control gives us low-level control over the trajectory sent to the robot. For more complicated motions, it is often useful to instead specify a goal and allow a motion planner to determine how the robot should reach it.

MoveIt is a motion-planning framework for robotic manipulators. In this part of the lab, you will build an action server that uses MoveIt's `/compute_cartesian_path` service to plan a **straight-line Cartesian path** for `tool0`, then executes that path on the robot.

Action Servers

Actions are a higher level ROS2 communication pattern designed for longer-running goals that may take a long time to complete, report back with progress updates, and be cancelled. An action interface has three parts:

1. Goal: what the client is asking the server to do
2. Feedback: progress/status updates
3. Result: the final outcome.

The high-level lifecycle is as follows:

1. A client sends a goal to an action server

2. The server decides whether to accept or reject the goal
3. If accepted, the server begins executing the goal and optionally publishing feedback
4. The client can optionally request cancellation. The server decides whether to accept the cancel request, and stops execution cleanly if so.
5. Once finished, the server returns a result (success, failure, or canceled).

To actually implement this, an action server has three callbacks:

- `goal_callback`, called when a new goal arrives. Return `GoalResponse.ACCEPT` or `GoalResponse.REJECT`. Accepting a goal causes `execute_callback` to run; rejecting it does not.
- `cancel_callback`, called when a client requests cancellation. Return `CancelResponse.ACCEPT` or `CancelResponse.REJECT`. Accepting marks the goal as cancel-requested; the `execute_callback` must check `goal_handle.is_cancel_requested` and actually stop.
- `execute_callback` performs the work, publishes feedback, and wraps up by calling `goal_handle.succeed()`, `goal_handle.abort()`, or `goal_handle.canceled()` and returning a result message.

The types of all the data used in these callbacks are defined in an `.action` file, just like a `.msg` or `.srv`. The action we'll be implementing in this section of the lab is described in `straight_line_interface/action/MoveStraight.action`.

```
# Goal
geometry_msgs/Pose target
float64 max_step
---
# Result
bool success
float64 planned_fraction
float64 dist_to_go
string message
---
# Feedback
float64 planned_fraction
string state
```

The goal contains a target `geometry_msgs/Pose` and a `max_step` (Cartesian step size for planning). The result reports `success`, how much of the path was planned (`planned_fraction`), remaining distance (`dist_to_go`), and a status `message`. Feedback reports `planned_fraction` and a string `state`, which we'll use to indicate what step we're currently on. You will learn more about action servers in Labs 4 and 5.

Task 4: Complete the TODOs in `straight_line_server.py`.

To test, make sure your server and MoveIt are running:

```
ros2 run straight_line straight_line_server
ros2 launch ur_moveit_config ur_moveit.launch.py ur_type:=ur7e launch_rviz:=true
```

Then, use `ros2 action` to send a goal:

```
ros2 action send_goal /move_straight \
straight_line_interface/action/MoveStraight \
"{target: {position: {x: 0.3, y: 0.1, z: 0.4}}, max_step: 0.01}" \
--feedback
```

Try changing `max_step`. This value determines the maximum Cartesian distance between consecutive points along the planned path. How does changing it affect the generated trajectory?

4.4 Conclusion

In this section, you moved the UR7e between the same two configurations using two different approaches. With direct joint control, the robot followed a trajectory defined in **joint space**, where the joint angles were interpolated from their starting values to their target values. Although this is a simple way to command the robot, a straight-line path in joint space does not generally correspond to a straight-line path of the end effector in the physical workspace. Using MoveIt, we instead planned a **straight-line Cartesian trajectory**. In this case, the desired path was defined in the workspace, and MoveIt computed the corresponding joint motion required for the end effector to follow that path.

This illustrates an important distinction in robot motion planning: we can describe motion either in terms of how the robot's joints should move or in terms of how we want the robot to move through the physical world.

Joint-Space Motion	Cartesian Motion
Straight path in joint space $\theta_A \rightarrow \theta_B$	Straight path in workspace $A \rightarrow B$
End-effector path may curve	Joint motion may be nonlinear

Checkpoint 3

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Demonstrate moving the UR7e between Pose A and Pose B using both direct joint control and MoveIt
 - Explain how your `JointTrajectory` command is constructed and sent to the robot
 - Compare/contrast the path taken by the robot when using the direct controller vs MoveIt
-

5 Before you Leave

Before finishing the lab, push your completed `lab3` directory to your personal GitHub repository:

```
cd ~/ros_workspaces
git add -A lab3
git commit -m "Complete Lab 3"
git push origin main
```

The lab workstations and arms are shared. Before you go:

- `Ctrl+C` out of every terminal. Do **not** use `Ctrl+Z` — that only pauses a process, and it keeps running in the background.
- `exit` all your distrobox sessions, and log out of your `ee106a-*` account.

Congratulations on finishing Lab 3!